Separate Files Code Marked

Main.cpp

```
#include "WeatherAnalyzer.h"  // Include the WeatherAnalyzer class header file
*/own code start
// Main function - Entry point of the program
int main()
{
    // Create an instance of the WeatherAnalyzer class
    WeatherAnalyzer analyzer;

    // Run the main workflow of the WeatherAnalyzer
    analyzer.run();

    // Return 0 to indicate successful execution
    return 0;
}
/*own code end
```

Candlestick.cpp

```
#include "Candlestick.h"
#include <iostream>
#include <iomanip>

*/own code start

// Method to display candlestick data in a formatted row with specific column widths
void Candlestick::displayRow() const
{
```

```
    // Output the date and candlestick values (open, close, high, low) with
proper spacing using setw
    std::cout << std::setw(12) << date
              << std::setw(10) << open
              << std::setw(10) << close
              << std::setw(10) << high
              << std::setw(10) << low
              << std::endl;
}
/*own code end
```

Candlestick.h

```
#ifndef CANDLESTICK_H
#define CANDLESTICK_H
#include <iostream>
#include <string>

*/own code start

// The Candlestick class represents a candlestick used for technical
analysis in weather data
class Candlestick
{
public:
    std::string date; // Date of the candlestick
    double open;      // Opening temperature for the time frame
    double high;      // Highest temperature for the time frame
    double low;       // Lowest temperature for the time frame
    double close;     // Closing temperature for the time frame

    // Default constructor initializing values to zero
    Candlestick() : open(0), high(0), low(0), close(0) {}
```

```
    // Parameterized constructor to initialize candlestick data with
specific values
    Candlestick(const std::string &date, double open, double high, double
low, double close)
        : date(date), open(open), high(high), low(low), close(close) {}

    // Method to display candlestick data in a formatted row (for debugging
purposes)
    void displayRow() const;
};

#endif // CANDLESTICK_H
/*own code end
```

CandlestickCalculator.cpp

```
#include "CandlestickCalculator.h"
#include <algorithm>
#include <numeric>

// Function to calculate daily candlestick data from hourly temperature
data.
// The function processes 24 hourly values at a time to compute a
candlestick for each day.
std::vector<Candlestick>
CandlestickCalculator::calculateDailyCandlesticks(const std::vector<double>
&hourlyData)
{
    std::vector<Candlestick> dailyCandlesticks;
    double previousClose = 0.0;  // Initialize the previous day's closing
value to 0.0

    // Iterate through the hourly data in 24-hour chunks
    for (size_t i = 0; i < hourlyData.size(); i += 24)
    {
```

```
        Candlestick candle;

        // The "Open" value is the previous day's average temperature
        candle.open = previousClose;

        // The "High" value is the maximum temperature of the day
        candle.high = *std::max_element(hourlyData.begin() + i,
hourlyData.begin() + std::min(i + 24, hourlyData.size()));

        // The "Low" value is the minimum temperature of the day
        candle.low = *std::min_element(hourlyData.begin() + i,
hourlyData.begin() + std::min(i + 24, hourlyData.size()));

        // The "Close" value is the average temperature of the day
        candle.close = std::accumulate(hourlyData.begin() + i,
hourlyData.begin() + std::min(i + 24, hourlyData.size()), 0.0) / 24;

        // Add the candlestick to the daily candlesticks vector
        dailyCandlesticks.push_back(candle);

        // Update the previous close value for the next day's open
        previousClose = candle.close;
    }
    return dailyCandlesticks;
}
/*own code start

// Function to calculate monthly candlestick data from daily candlestick
data.
// The function processes 30 daily candlesticks at a time to compute a
candlestick for each month.
std::vector<Candlestick>
CandlestickCalculator::calculateMonthlyCandlesticks(const
std::vector<Candlestick> &dailyCandlesticks)
{
    std::vector<Candlestick> monthlyCandlesticks;
```

```
    double previousClose = 0.0;  // Initialize the previous month's closing
value to 0.0


    // Iterate through the daily candlesticks in 30-day chunks

    for (size_t i = 0; i < dailyCandlesticks.size(); i += 30)

    {

        Candlestick candle;


        // The "Open" value is the previous month's average temperature

        candle.open = previousClose;


        // The "High" value is the highest temperature recorded in the
month

        candle.high = std::max_element(dailyCandlesticks.begin() + i,
dailyCandlesticks.begin() + std::min(i + 30, dailyCandlesticks.size()),

                                       [](const Candlestick &a, const
Candlestick &b) { return a.high < b.high; })

                           ->high;


        // The "Low" value is the lowest temperature recorded in the month

        candle.low = std::min_element(dailyCandlesticks.begin() + i,
dailyCandlesticks.begin() + std::min(i + 30, dailyCandlesticks.size()),

                                      [](const Candlestick &a, const
Candlestick &b) { return a.low < b.low; })

                          ->low;


        // The "Close" value is the average temperature of the month

        candle.close = std::accumulate(dailyCandlesticks.begin() + i,
dailyCandlesticks.begin() + std::min(i + 30, dailyCandlesticks.size()),
0.0,

                                       [](double sum, const Candlestick &c)
{ return sum + c.close; }) / 30;


        // Add the candlestick to the monthly candlesticks vector

        monthlyCandlesticks.push_back(candle);
```

```
        // Update the previous close value for the next month's open
        previousClose = candle.close;
    }
    return monthlyCandlesticks;
}

// Function to calculate yearly candlestick data from monthly candlestick data.
// The function processes 12 monthly candlesticks at a time to compute a candlestick for each year.
std::vector<Candlestick> CandlestickCalculator::calculateYearlyCandlesticks(const std::vector<Candlestick> &monthlyCandlesticks)
{
    std::vector<Candlestick> yearlyCandlesticks;
    double previousClose = 0.0;  // Initialize the previous year's closing value to 0.0

    // Iterate through the monthly candlesticks in 12-month chunks
    for (size_t i = 0; i < monthlyCandlesticks.size(); i += 12)
    {
        Candlestick candle;

        // The "Open" value is the previous year's average temperature
        candle.open = previousClose;

        // The "High" value is the highest temperature recorded in the year
        candle.high = std::max_element(monthlyCandlesticks.begin() + i, monthlyCandlesticks.begin() + std::min(i + 12, monthlyCandlesticks.size()),
                                       [](const Candlestick &a, const Candlestick &b) { return a.high < b.high; })
                          ->high;

        // The "Low" value is the lowest temperature recorded in the year
        candle.low = std::min_element(monthlyCandlesticks.begin() + i, monthlyCandlesticks.begin() + std::min(i + 12, monthlyCandlesticks.size()),
```

```
                                        [](const Candlestick &a, const
Candlestick &b) { return a.low < b.low; })
                            ->low;

        // The "Close" value is the average temperature of the year
        candle.close = std::accumulate(monthlyCandlesticks.begin() + i,
monthlyCandlesticks.begin() + std::min(i + 12, monthlyCandlesticks.size()),
0.0,
                                        [](double sum, const Candlestick &c)
{ return sum + c.close; }) / 12;

        // Add the candlestick to the yearly candlesticks vector
        yearlyCandlesticks.push_back(candle);

        // Update the previous close value for the next year's open
        previousClose = candle.close;
    }
    return yearlyCandlesticks;
}

/*own code end
```

CandlestickCalculator.h

```
#ifndef CANDLESTICKCALCULATOR_H
#define CANDLESTICKCALCULATOR_H

#include <vector>
#include "Candlestick.h"
*/own code start

// The CandlestickCalculator class provides methods to calculate
candlestick data from temperature readings
class CandlestickCalculator
{
```

```
public:

    // Method to calculate daily candlestick data from hourly temperature
readings

    static std::vector<Candlestick> calculateDailyCandlesticks(const
std::vector<double> &hourlyData);


    // Method to calculate monthly candlestick data from daily candlesticks

    static std::vector<Candlestick> calculateMonthlyCandlesticks(const
std::vector<Candlestick> &dailyCandlesticks);


    // Method to calculate yearly candlestick data from monthly
candlesticks

    static std::vector<Candlestick> calculateYearlyCandlesticks(const
std::vector<Candlestick> &monthlyCandlesticks);

};


#endif // CANDLESTICKCALCULATOR_H

/*own code end
```

CandlestickPlot.cpp

```
// CandlestickPlot.cpp

#include "CandlestickPlot.h"

#include <iostream>

#include <iomanip>


// Function to plot the weather candlesticks as a text-based chart

// Parameters:

// - candlesticks: A vector containing candlestick objects to be plotted

// - chartHeight: The height of the text chart in lines

// - frequency: The time period frequency ('d' for daily, 'm' for monthly,
'y' for yearly)

*/own code start
```

```
void CandlestickPlot::plot(const std::vector<Candlestick> &candlesticks,
int chartHeight, char frequency)

{

    // Check if the input candlestick data is empty

    if (candlesticks.empty())

    {

        std::cerr << "No data to plot." << std::endl;

        return;

    }


    // Initialize variables to track the minimum and maximum temperature
values for the Y-axis

    double minTemp = candlesticks.front().low;

    double maxTemp = candlesticks.front().high;


    // Loop through all candlesticks to find the overall minimum and
maximum temperatures

    for (const auto &candle : candlesticks)

    {

        if (candle.low < minTemp)

            minTemp = candle.low;

        if (candle.high > maxTemp)

            maxTemp = candle.high;

    }


    // Calculate the range of temperatures and the step size for each row
in the chart

    double range = maxTemp - minTemp;

    double step = range / chartHeight;


    // Print the header of the chart

    std::cout << "\n====================== Weather Candlestick Chart
======================\n";


    // Loop to generate the plot row by row
```

```
    for (int i = chartHeight; i >= 0; --i)
    {
        // Calculate the current temperature for this row
        double currentTemp = minTemp + i * step;

        // Print the Y-axis label with temperature in °C
        std::cout << std::setw(6) << std::fixed << std::setprecision(1) <<
currentTemp << "°C | ";

        // Loop through each candlestick to plot the corresponding symbols
for high, low, open, and close values
        for (size_t j = 0; j < candlesticks.size(); ++j)
        {
            const auto &candle = candlesticks[j];

            // Add spacing between candlesticks for better readability
            std::cout << std::setw(5);

            // Determine which symbol to plot based on the current
temperature and candlestick values
            if (std::abs(currentTemp - candle.high) < step / 2.0)
            {
                std::cout << " | "; // High stalk
            }
            else if (std::abs(currentTemp - candle.low) < step / 2.0)
            {
                std::cout << " | "; // Low stalk
            }
            else if (std::abs(currentTemp - candle.open) < step / 2.0)
            {
                std::cout << "----"; // Open value
            }
            else if (std::abs(currentTemp - candle.close) < step / 2.0)
            {
```

```
                std::cout << "----"; // Close value
            }
            else if (currentTemp < candle.high && currentTemp > candle.low)
            {
                std::cout << " | "; // Inside stalk
            }
            else
            {
                std::cout << "     "; // Empty space
            }
        }

        // Move to the next row
        std::cout << std::endl;
    }

    // Print the X-axis labels based on the selected frequency (daily,
monthly, yearly)
    if (frequency == 'd')
    {
        printDailyXAxis(candlesticks);
    }
    else if (frequency == 'm')
    {
        printMonthlyXAxis(candlesticks);
    }
    else if (frequency == 'y')
    {
        printYearlyXAxis(candlesticks);
    }

    // Note about the representation of open/close values
    std::cout << "\nNote:\n";
```

```
    std::cout << "If Open and Close values are closer than " << step / 2.0
              << "°C (based on the current Y-axis resolution), they are
represented by a single horizontal line ('----').\n";
}

// Helper function to print daily X-axis labels
void CandlestickPlot::printDailyXAxis(const std::vector<Candlestick>
&candlesticks)
{
    // Extract the start day from the first candlestick's date
    int startDay = std::stoi(candlesticks.front().date.substr(8, 2));

    // Print day numbers with proper spacing
    std::cout << "       ";
    for (size_t i = 0; i < candlesticks.size(); ++i)
    {
        std::cout << std::setw(5) << (startDay + i);
    }
    std::cout << "\n       ";

    // Print the month number as a label in the middle of the X-axis
    if (!candlesticks.empty())
    {
        std::cout << std::setw(5 * candlesticks.size() / 2) <<
candlesticks.front().date.substr(5, 2) << "\n";
    }
}

// Helper function to print monthly X-axis labels
void CandlestickPlot::printMonthlyXAxis(const std::vector<Candlestick>
&candlesticks)
{
    // Extract the start month from the first candlestick's date
    int startMonth = std::stoi(candlesticks.front().date.substr(5, 2));
```

```
    // Print month numbers with proper spacing
    std::cout << "       ";
    for (size_t i = 0; i < candlesticks.size(); ++i)
    {
        std::cout << std::setw(5) << ((startMonth + i - 1) % 12 + 1);
    }
    std::cout << "\n       ";

    // Print the year labels at the appropriate positions
    for (size_t i = 0; i < candlesticks.size(); ++i)
    {
        if ((startMonth + i - 1) % 12 == 0)
        {
            std::cout << std::setw(5) << candlesticks[i].date.substr(0, 4);
        }
        else
        {
            std::cout << "     ";
        }
    }
    std::cout << "\n";
}

// Helper function to print yearly X-axis labels
void CandlestickPlot::printYearlyXAxis(const std::vector<Candlestick>
&candlesticks)
{
    // Extract the start year from the first candlestick's date
    int startYear = std::stoi(candlesticks.front().date.substr(0, 4));

    // Print year numbers with proper spacing
    std::cout << "       ";
    for (size_t i = 0; i < candlesticks.size(); ++i)
```

```
    {
        std::cout << std::setw(5) << (startYear + i);
    }
    std::cout << "\n";
}
/*own code end
```

CandlestickPlot.h

```
// CandlestickPlot.h
#ifndef CANDLESTICKPLOT_H
#define CANDLESTICKPLOT_H

#include <vector>
#include "Candlestick.h"
*/own code start

// The CandlestickPlot class provides methods to plot candlestick data using text-based charts
class CandlestickPlot
{
public:
    // Function to plot the weather candlesticks as a text-based chart
    // Parameters:
    // - candlesticks: A vector of Candlestick objects to be plotted
    // - chartHeight: The height of the text chart in lines
    // - frequency: The time period frequency ('d' for daily, 'm' for monthly, 'y' for yearly)
    static void plot(const std::vector<Candlestick> &candlesticks, int chartHeight, char frequency);

private:
```

```
    // Helper function to print daily X-axis labels on the chart
    // Parameters:
    // - candlesticks: A vector of Candlestick objects used to determine
the labels
    static void printDailyXAxis(const std::vector<Candlestick>
&candlesticks);

    // Helper function to print monthly X-axis labels on the chart
    // Parameters:
    // - candlesticks: A vector of Candlestick objects used to determine
the labels
    static void printMonthlyXAxis(const std::vector<Candlestick>
&candlesticks);

    // Helper function to print yearly X-axis labels on the chart
    // Parameters:
    // - candlesticks: A vector of Candlestick objects used to determine
the labels
    static void printYearlyXAxis(const std::vector<Candlestick>
&candlesticks);
};

#endif // CANDLESTICKPLOT_H
/*own code end
```

DataParser.cpp

```
// DataParser.cpp
#include "DataParser.h"
#include <fstream>
#include <sstream>
#include <iostream>
#include <algorithm>
*/own code start
```

```
// Constructor that initializes the file path and country code for the DataParser
DataParser::DataParser(const std::string &filePath, const std::string &countryCode)
    : filePath(filePath), countryCode(countryCode) {}

// Method to retrieve the parsed temperature data in candlestick format
std::vector<Candlestick> DataParser::getTemperatureData() const
{
    // Return the previously computed candlestick data
    return candlestickData;
}

// Method to parse the CSV file and extract hourly temperature data for the given country code
void DataParser::parse()
{
    // Open the CSV file for reading
    std::ifstream file(filePath);
    if (!file.is_open())
    {
        std::cerr << "Error opening file: " << filePath << std::endl;
        return;
    }

    // Read the header line to find the relevant country column
    std::string line, header;
    std::getline(file, header);

    // Use a string stream to parse the header line
    std::istringstream headerStream(header);
    std::string column;
    int countryIndex = -1;
    int index = 0;
```

```
    // Find the column index for the specified country code
    while (std::getline(headerStream, column, ','))
    {
        if (column == countryCode + "_temperature")
        {
            countryIndex = index;
            break;
        }
        ++index;
    }

    // If the country code is not found in the header, print an error
message and return
    if (countryIndex == -1)
    {
        std::cerr << "Country code not found in file header." << std::endl;
        return;
    }

    // Read the data lines from the CSV file
    while (std::getline(file, line))
    {
        // Use a string stream to parse each line
        std::istringstream lineStream(line);
        std::string date, value;
        std::vector<std::string> values;

        // Split the line by commas and store the values in a vector
        while (std::getline(lineStream, value, ','))
        {
            values.push_back(value);
        }
```

```
        // If the vector size is greater than the country index, extract
the temperature data
        if (values.size() > static_cast<size_t>(countryIndex))
        {
            std::string datetime = values[0];                           //
The first value is the date and time
            double temperature = std::stod(values[countryIndex]);       //
Convert the temperature value to double
            hourlyData[datetime.substr(0, 10)].push_back(temperature); //
Group the temperature data by day
        }
    }

    // Close the CSV file
    file.close();
}

// Method to compute daily candlestick data from the parsed hourly data
std::vector<Candlestick> DataParser::computeDailyCandlesticks()
{
    std::vector<Candlestick> candlesticks;
    double previousClose = 0.0; // Store the closing value of the previous
day

    // Iterate through each day's hourly data
    for (const auto &entry : hourlyData)
    {
        const std::vector<double> &temps = entry.second;             // Get
the temperatures for the day
        Candlestick candle;                                          //
Create a new candlestick object
        candle.date = entry.first;                                   // Set
the date of the candlestick
        candle.open = previousClose;                                 // Set
the open value to the previous day's close
```

```
        candle.high = *std::max_element(temps.begin(), temps.end()); // Find the highest temperature
        candle.low = *std::min_element(temps.begin(), temps.end());  // Find the lowest temperature
        candle.close = temps.back();                                 // Set the close value to the last temperature of the day

        previousClose = candle.close;   // Update the previous close value
        candlesticks.push_back(candle); // Add the candlestick to the vector
    }
    return candlesticks;
}

// Method to compute monthly candlestick data from daily candlestick data
std::vector<Candlestick> DataParser::computeMonthlyCandlesticks()
{
    std::map<std::string, std::vector<double>> monthlyData; // Map to store temperature data grouped by month

    // Group the daily temperature data by month
    for (const auto &entry : hourlyData)
    {
        std::string month = entry.first.substr(0, 7); // Extract the year and month (YYYY-MM)
        monthlyData[month].insert(monthlyData[month].end(), entry.second.begin(), entry.second.end());
    }

    std::vector<Candlestick> candlesticks;
    double previousClose = 0.0; // Store the closing value of the previous month

    // Iterate through each month's temperature data
    for (const auto &entry : monthlyData)
    {
```

```
        const std::vector<double> &temps = entry.second;             // Get
the temperatures for the month

        Candlestick candle;                                          //
Create a new candlestick object

        candle.date = entry.first + "-01";                           // Set
the date of the candlestick to the first day of the month

        candle.open = previousClose;                                 // Set
the open value to the previous month's close

        candle.high = *std::max_element(temps.begin(), temps.end()); //
Find the highest temperature

        candle.low = *std::min_element(temps.begin(), temps.end());  //
Find the lowest temperature

        candle.close = temps.back();                                 // Set
the close value to the last temperature of the month


        previousClose = candle.close;   // Update the previous close value

        candlesticks.push_back(candle); // Add the candlestick to the
vector

    }

    return candlesticks;

}

/*own code end


// Method to compute yearly candlestick data from monthly candlestick data

std::vector<Candlestick> DataParser::computeYearlyCandlesticks()

{

    std::map<std::string, std::vector<double>> yearlyData; // Map to store
temperature data grouped by year


    // Group the monthly temperature data by year

    for (const auto &entry : hourlyData)

    {

        std::string year = entry.first.substr(0, 4); // Extract the year
(YYYY)

        yearlyData[year].insert(yearlyData[year].end(),
entry.second.begin(), entry.second.end());

    }
```

```
    std::vector<Candlestick> candlesticks;

    double previousClose = 0.0; // Store the closing value of the previous
year


    // Iterate through each year's temperature data

    for (const auto &entry : yearlyData)

    {

        const std::vector<double> &temps = entry.second;             // Get
the temperatures for the year

        Candlestick candle;                                          //
Create a new candlestick object

        candle.date = entry.first + "-01-01";                        // Set
the date of the candlestick to the first day of the year

        candle.open = previousClose;                                 // Set
the open value to the previous year's close

        candle.high = *std::max_element(temps.begin(), temps.end()); //
Find the highest temperature

        candle.low = *std::min_element(temps.begin(), temps.end());  //
Find the lowest temperature

        candle.close = temps.back();                                 // Set
the close value to the last temperature of the year


        previousClose = candle.close;   // Update the previous close value

        candlesticks.push_back(candle); // Add the candlestick to the
vector

    }

    return candlesticks;

}
```

DataParser.h

```
// DataParser.h

#ifndef DATAPARSER_H

#define DATAPARSER_H


#include <string>
```

```
#include <map>

#include <vector>

#include "Candlestick.h"


// The DataParser class is responsible for parsing temperature data from a
CSV file

// and computing candlestick data (daily, monthly, and yearly)

*/own code start

class DataParser

{

private:

    std::string filePath;                                    // Path to the
CSV file containing weather data

    std::string countryCode;                                 // Country code
to filter the data (e.g., "GB" for Great Britain)

    std::map<std::string, std::vector<double>> hourlyData; // Map to store
hourly temperature data grouped by date

    std::vector<Candlestick> candlestickData;              // Vector to
store computed candlestick data


public:

    // Constructor to initialize the file path and country code

    // Parameters:

    // - filePath: Path to the CSV file

    // - countryCode: Two-character country code for filtering the data

    DataParser(const std::string &filePath, const std::string
&countryCode);


    // Method to parse the CSV file and extract hourly temperature data for
the specified country

    void parse();


    // Method to compute daily candlestick data from the parsed hourly data

    // Returns: A vector of Candlestick objects representing daily
candlesticks

    std::vector<Candlestick> computeDailyCandlesticks();
```

```
    // Method to compute monthly candlestick data from daily candlestick
data

    // Returns: A vector of Candlestick objects representing monthly
candlesticks

    std::vector<Candlestick> computeMonthlyCandlesticks();


    // Method to compute yearly candlestick data from monthly candlestick
data

    // Returns: A vector of Candlestick objects representing yearly
candlesticks

    std::vector<Candlestick> computeYearlyCandlesticks();


    // Method to retrieve the parsed temperature data in candlestick format

    // Returns: A vector of Candlestick objects representing the
temperature data

    std::vector<Candlestick> getTemperatureData() const;

};


#endif // DATAPARSER_H


/*own code end
```

Filters.cpp

```
// Filters.cpp

#include "Filters.h"

#include <algorithm>


// Function to filter candlestick data by a specified date range

// Parameters:

// - candlesticks: A vector of candlesticks to be filtered

// - startDate: The start date of the desired range (inclusive)

// - endDate: The end date of the desired range (exclusive)
```

```
std::vector<Candlestick> Filters::filterByDateRange(const
std::vector<Candlestick> &candlesticks, const std::string &startDate, const
std::string &endDate)

{

    std::vector<Candlestick> filtered; // Vector to store filtered data


    // Iterate over each candlestick to check if its date falls within the
specified range

    for (const auto &candle : candlesticks)

    {

        if (candle.date >= startDate && candle.date < endDate)

        {

            filtered.push_back(candle); // Add the candlestick to the
filtered list if it matches the criteria

        }

    }

    return filtered; // Return the filtered vector

}

*/own code start


// Function to filter candlestick data by a specific month across all years

// Parameters:

// - candlesticks: A vector of candlesticks to be filtered

// - targetMonth: The month to filter by (in MM format)

std::vector<Candlestick> Filters::filterByMonthAcrossYears(const
std::vector<Candlestick> &candlesticks, const std::string &targetMonth)

{

    std::vector<Candlestick> filtered; // Vector to store filtered data


    // Iterate over each candlestick to check if its date contains the
target month

    for (const auto &candle : candlesticks)

    {

        if (candle.date.substr(5, 2) == targetMonth)

        {
```

```
            filtered.push_back(candle); // Add the candlestick to the
filtered list if it matches the criteria
        }
    }
    return filtered; // Return the filtered vector
}

// Function to filter candlestick data by a minimum temperature value
// Parameters:
// - candlesticks: A vector of candlesticks to be filtered
// - minTemp: The minimum temperature to filter by
std::vector<Candlestick> Filters::filterByMinTemperature(const
std::vector<Candlestick> &candlesticks, double minTemp)
{
    std::vector<Candlestick> filtered; // Vector to store filtered data

    // Iterate over each candlestick to check if its low value is greater
than or equal to the minimum temperature
    for (const auto &candle : candlesticks)
    {
        if (candle.low >= minTemp)
        {
            filtered.push_back(candle); // Add the candlestick to the
filtered list if it matches the criteria
        }
    }
    return filtered; // Return the filtered vector
}

// Function to filter candlestick data by a maximum temperature value
// Parameters:
// - candlesticks: A vector of candlesticks to be filtered
// - maxTemp: The maximum temperature to filter by
std::vector<Candlestick> Filters::filterByMaxTemperature(const
std::vector<Candlestick> &candlesticks, double maxTemp)
```

```
{
    std::vector<Candlestick> filtered; // Vector to store filtered data

    // Iterate over each candlestick to check if its high value is less than or equal to the maximum temperature
    for (const auto &candle : candlesticks)
    {
        if (candle.high <= maxTemp)
        {
            filtered.push_back(candle); // Add the candlestick to the filtered list if it matches the criteria
        }
    }
    return filtered; // Return the filtered vector
}

// Function to filter candlestick data by a temperature range
// Parameters:
// - candlesticks: A vector of candlesticks to be filtered
// - minTemp: The minimum temperature of the range
// - maxTemp: The maximum temperature of the range
std::vector<Candlestick> Filters::filterByTemperatureRange(const std::vector<Candlestick> &candlesticks, double minTemp, double maxTemp)
{
    std::vector<Candlestick> filtered; // Vector to store filtered data

    // Iterate over each candlestick to check if its low and high values fall within the specified range
    for (const auto &candle : candlesticks)
    {
        if (candle.low >= minTemp && candle.high <= maxTemp)
        {
            filtered.push_back(candle); // Add the candlestick to the filtered list if it matches the criteria
        }
```

```
    }

    return filtered; // Return the filtered vector

}


// Function to return all candlestick data without applying any filters

// Parameters:

// - candlesticks: A vector of candlesticks to be returned as is

std::vector<Candlestick> Filters::noFilter(const std::vector<Candlestick>
&candlesticks)

{

    return candlesticks; // Return the input vector without any
modifications

}

/*own code end
```

Filters.h

```
#ifndef FILTERS_H

#define FILTERS_H


#include <vector>

#include <string>

#include "Candlestick.h"

*/own code start


class Filters

{

public:


    // Filter by date range

    static std::vector<Candlestick> filterByDateRange(const
std::vector<Candlestick> &candlesticks, const std::string &startDate, const
std::string &endDate);


    // Filter by month across all years
```

```
    static std::vector<Candlestick> filterByMonthAcrossYears(const
std::vector<Candlestick> &candlesticks, const std::string &targetMonth);


    // Filter by minimum temperature

    static std::vector<Candlestick> filterByMinTemperature(const
std::vector<Candlestick> &candlesticks, double minTemp);


    // Filter by maximum temperature

    static std::vector<Candlestick> filterByMaxTemperature(const
std::vector<Candlestick> &candlesticks, double maxTemp);


    // Filter by temperature range

    static std::vector<Candlestick> filterByTemperatureRange(const
std::vector<Candlestick> &candlesticks, double minTemp, double maxTemp);


    // No filter (return all data)

    static std::vector<Candlestick> noFilter(const std::vector<Candlestick>
&candlesticks);




};


#endif // FILTERS_H

/*own code end
```

Predictor.cpp

```
// Predictor.cpp

#include "Predictor.h"

#include <numeric>

*/own code start


// Function to predict future temperatures using a moving average method

// Parameters:

// - candlesticks: A vector of historical candlestick data
```

```
// - windowSize: The size of the moving average window (number of periods to consider for prediction)

// - futurePeriods: The number of future periods to predict

// - periodType: The type of period ('d' for daily, 'm' for monthly, 'y' for yearly)

std::vector<Candlestick> predictFutureTemperatures(const std::vector<Candlestick> &candlesticks, int windowSize, int futurePeriods, char periodType)

{

    std::vector<Candlestick> predictions;                          // Vector to store predicted candlestick data

    std::vector<Candlestick> workingCandlesticks = candlesticks; // Copy of the original data to work with


    // If there isn't enough data for the given window size, return an empty vector

    if (candlesticks.size() < windowSize)

    {

        return predictions;

    }


    // Determine the increment for calculating the next date based on the period type

    int increment = (periodType == 'd') ? 1 : (periodType == 'm') ? 30 : 365;


    // Get the last date from the historical dataset

    std::string lastDate = candlesticks.back().date;

    int lastYear = std::stoi(lastDate.substr(0, 4));

    int lastMonth = std::stoi(lastDate.substr(5, 2));

    int lastDay = std::stoi(lastDate.substr(8, 2));


    // Loop to predict the specified number of future periods

    for (int i = 0; i < futurePeriods; ++i)

    {

        double sumHigh = 0, sumLow = 0, sumOpen = 0, sumClose = 0;
```

```
        // Calculate the moving averages for the last `windowSize` periods
        for (size_t j = workingCandlesticks.size() - windowSize; j <
workingCandlesticks.size(); ++j)
        {
            sumHigh += workingCandlesticks[j].high;
            sumLow += workingCandlesticks[j].low;
            sumOpen += workingCandlesticks[j].open;
            sumClose += workingCandlesticks[j].close;
        }

        // Compute the average values for the predicted candlestick
        double avgHigh = sumHigh / windowSize;
        double avgLow = sumLow / windowSize;
        double avgOpen = sumOpen / windowSize;
        double avgClose = sumClose / windowSize;

        // Calculate the next date for the prediction
        lastDay += increment;
        if (lastDay > 30)
        {
            lastDay = 1;
            ++lastMonth;
        }
        if (lastMonth > 12)
        {
            lastMonth = 1;
            ++lastYear;
        }

        // Format the predicted date as a string (YYYY-MM-DD)
        std::string predictedDate = std::to_string(lastYear) + "-" +
                                    (lastMonth < 10 ? "0" : "") +
std::to_string(lastMonth) + "-" +
```

```
                                    (lastDay < 10 ? "0" : "") +
std::to_string(lastDay);


        // Create a new candlestick object with the predicted values

        predictions.emplace_back(predictedDate, avgOpen, avgHigh, avgLow,
avgClose);


        // Add the new prediction to the working dataset for subsequent
predictions

        workingCandlesticks.push_back(predictions.back());

    }


    // Return the vector of predicted candlesticks

    return predictions;

}

/*own code end
```

Predictor.h

```
#ifndef PREDICTOR_H

#define PREDICTOR_H


#include "Candlestick.h"

#include <vector>

*/own code start


// Function to predict future temperatures

std::vector<Candlestick> predictFutureTemperatures(const
std::vector<Candlestick> &candlesticks, int windowSize, int futurePeriods,
char periodType);


#endif // PREDICTOR_H

/*own code end
```

WeatherAnalyzer.cpp

```
#include "WeatherAnalyzer.h"
*/own code start

// No additional implementation needed since all methods are defined in the header
/*own code end
```

WeatherAnalyzer.h

```
#ifndef WEATHERANALYZER_H
#define WEATHERANALYZER_H

#include <iostream>          // For standard input/output operations
#include <string>            // For using the string class
#include <vector>            // For using vectors to store data
#include <map>               // For using maps to store key-value pairs
#include <iomanip>           // For formatted output
#include <numeric>           // For numeric operations such as accumulate
#include <algorithm>         // For using std::max_element and std::min_element functions
#include "DataParser.h"      // Header file for data parsing
#include "CandlestickPlot.h" // Header file for plotting candlestick charts
#include "Filters.h"         // Header file for filtering candlestick data
#include "Predictor.h"       // Header file for predicting future temperature data
*/own code start

class WeatherAnalyzer
{
public:
    // This function lists the available country codes from the dataset for the user's reference
    void displayAvailableCountries()
    {
        std::cout << "Total 350640 entries\n";
```

```
        std::cout << "Country codes available: [ AT BE BG CH CZ DE DK EE ES FI FR GB GR HR HU IE IT LT LU LV NL NO PL PT RO SE SI SK ]\n";
    }

    // Function to prompt the user for a date input and return it as a string
    // Parameters:
    // - prompt: The message to display when asking for input
    std::string promptDate(const std::string &prompt)
    {
        std::string date; // Variable to store the date input
        std::cout << prompt;
        std::cin >> date; // Get the user input
        return date;      // Return the entered date
    }
/*own code end

    // Function to prompt the user to choose a frequency for plotting (daily, monthly, or yearly)
    // Returns the frequency character entered by the user

    char promptFrequency()
    {
        char freq; // Variable to store the frequency input
        std::cout << "Enter frequency (y(yearly), m(monthly), d(daily)): ";
        std::cin >> freq; // Get the user input
        return freq;      // Return the entered frequency
    }
*/own code start

    // Function to calculate and display the mean, maximum, and minimum temperatures from the candlestick data
    // Parameters:
    // - candlesticks: A vector of Candlestick objects containing temperature data
```

```
    void displayStats(const std::vector<Candlestick> &candlesticks)
    {
        // Calculate the mean temperature by summing the close values and
dividing by the number of entries
        double meanTemp = std::accumulate(candlesticks.begin(),
candlesticks.end(), 0.0,
                                          [](double sum, const Candlestick
&c) { return sum + c.close; }) /
                           candlesticks.size();

        // Find the maximum temperature by comparing the high values of all
candlesticks
        double maxTemp = std::max_element(candlesticks.begin(),
candlesticks.end(),
                                          [](const Candlestick &a, const
Candlestick &b) { return a.high < b.high; })
                              ->high;

        // Find the minimum temperature by comparing the low values of all
candlesticks
        double minTemp = std::min_element(candlesticks.begin(),
candlesticks.end(),
                                          [](const Candlestick &a, const
Candlestick &b) { return a.low < b.low; })
                              ->low;

        // Display the calculated statistics
        std::cout << "\n[**] Mean Temperature: " << meanTemp << "°C";
        std::cout << "\n[**] Max Temperature: " << maxTemp << "°C";
        std::cout << "\n[**] Min Temperature: " << minTemp << "°C\n";
    }

    // Function to plot multiple graphs with a limit of 30 entries per
graph
    // Parameters:
    // - candlesticks: A vector of Candlestick objects to be plotted
    // - chartHeight: The height of the chart to be displayed
```

```
    // - frequency: The frequency of the data (daily, monthly, or yearly)
    // Function to plot multiple graphs with a limit of 30 entries per graph
    // Displays only a final summary after all graphs are plotted
    void plotMultipleGraphs(const std::vector<Candlestick> &candlesticks, int chartHeight, char frequency)
    {
        const int maxEntriesPerGraph = 30;         // Maximum number of entries to display per graph
        size_t totalEntries = candlesticks.size(); // Total number of candlesticks

        // Plot all graphs in subsets of maxEntriesPerGraph
        size_t startIndex = 0;
        while (startIndex < totalEntries)
        {
            size_t endIndex = std::min(startIndex + maxEntriesPerGraph, totalEntries);
            std::vector<Candlestick> subCandlesticks(candlesticks.begin() + startIndex, candlesticks.begin() + endIndex);

            // Call the plot function to display the current subset of candlesticks
            CandlestickPlot::plot(subCandlesticks, chartHeight, frequency);

            startIndex += maxEntriesPerGraph;
        }

        // Display the final summary once after all graphs are plotted
        std::cout << "\n============================\n";
        std::cout << "Summary:\n";
        displayStats(candlesticks);

        std::cout << "\nLegend:\n";
        std::cout << "'|' : High/Low stalk\n";
        std::cout << "'----' : Open/Close values\n";
```

```
        std::cout << "\nNote:\n";
        std::cout << "The Open/Close values are represented as horizontal lines ('----'), and the High/Low values are shown as vertical lines ('|').\n";
    }

    // Function to apply additional filters to the candlestick data based on user input
    // Parameters:
    // - candlesticks: A vector of Candlestick objects to be filtered
    // - frequency: The frequency of the data (daily, monthly, or yearly)
    void applyAdditionalFilters(std::vector<Candlestick> &candlesticks, char frequency)
    {
        int choice; // Variable to store user's choice for filter options
        do
        {
            // Display filter options to the user
            std::cout << "\nSelect a filter option:\n";
            std::cout << "1. Filter by Month Across All Years\n";
            std::cout << "2. Filter by Minimum Temperature\n";
            std::cout << "3. Filter by Maximum Temperature\n";
            std::cout << "4. Filter by Temperature Range\n";
            std::cout << "5. No Filter (Display All Data)\n";
            std::cout << "Enter your choice: ";
            std::cin >> choice; // Get the user's choice

            std::vector<Candlestick> filteredCandlesticks; // Vector to store the filtered candlestick data

            // Apply the filter based on the user's choice
            switch (choice)
            {
            case 1:
```

```
            {
                // Filter by a specific month across all years
                std::string targetMonth;
                std::cout << "Enter month (MM): ";
                std::cin >> targetMonth;
                filteredCandlesticks =
Filters::filterByMonthAcrossYears(candlesticks, targetMonth);
                break;
            }
            case 2:
            {
                // Filter by minimum temperature
                double minTemp;
                std::cout << "Enter minimum temperature: ";
                std::cin >> minTemp;
                filteredCandlesticks =
Filters::filterByMinTemperature(candlesticks, minTemp);
                break;
            }
            case 3:
            {
                // Filter by maximum temperature
                double maxTemp;
                std::cout << "Enter maximum temperature: ";
                std::cin >> maxTemp;
                filteredCandlesticks =
Filters::filterByMaxTemperature(candlesticks, maxTemp);
                break;
            }
            case 4:
            {
                // Filter by a temperature range (minimum and maximum)
                double minTemp, maxTemp;
                std::cout << "Enter minimum temperature: ";
```

```
                std::cin >> minTemp;
                std::cout << "Enter maximum temperature: ";
                std::cin >> maxTemp;
                filteredCandlesticks =
Filters::filterByTemperatureRange(candlesticks, minTemp, maxTemp);
                break;
            }
            case 5:
                // No filter applied, return all data
                filteredCandlesticks = Filters::noFilter(candlesticks);
                break;
            default:
                // Handle invalid input
                std::cerr << "Invalid choice. Please try again.\n";
                continue;
            }

            // Display the filtered data in a table format
            std::cout << "\nOpen, High, Low, and Close values for the
filtered range:\n";
            std::cout << std::setw(12) << "Date"
                      << std::setw(12) << "Open/°C"
                      << std::setw(12) << "Close/°C"
                      << std::setw(12) << "High/°C"
                      << std::setw(12) << "Low/°C" << std::endl;

            // Iterate over the filtered candlesticks and display each row
            for (const auto &candle : filteredCandlesticks)
            {
                candle.displayRow();
            }

            // Plot the filtered data on a chart
            plotMultipleGraphs(filteredCandlesticks, 20, frequency);
```

```
            // Ask if the user wants to apply more filters
            char moreFilters;
            std::cout << "\nWould you like to apply more filters? (y/n): ";
            std::cin >> moreFilters;
            if (moreFilters == 'n' || moreFilters == 'N')
            {
                break; // Exit the loop if the user chooses not to apply
more filters
            }

        } while (true);
    }

    void run()
    {
        // Display the list of available countries to the user
        displayAvailableCountries();

        // Prompt the user to enter a country code
        std::string countryCode;
        std::cout << "Enter country code: ";
        std::cin >> countryCode;

        // Load the data from the specified CSV file for the chosen country
        std::string filePath = "weather_data_EU_1980-2019_temp_only.csv";
        DataParser parser(filePath, countryCode);
        parser.parse(); // Parse the data

        // === Task1 CODE STARTS HERE ===
        // Compute and display yearly candlestick data for the past 40
years
        std::vector<Candlestick> fullData =
parser.computeYearlyCandlesticks();
```

```
        std::cout << "\nDisplaying data for 40 years:\n";

        // Print table headers for the candlestick data
        std::cout << std::setw(12) << "Date"
                  << std::setw(12) << "Open/°C"
                  << std::setw(12) << "Close/°C"
                  << std::setw(12) << "High/°C"
                  << std::setw(12) << "Low/°C" << std::endl;

        // Iterate through the yearly candlestick data and display each row
        for (const auto &candle : fullData)
        {
            candle.displayRow();
        }

        // Plot the yearly candlestick data
        plotMultipleGraphs(fullData, 20, 'y');

        // === Task1 CODE ENDS HERE ===

        // Prompt the user to decide whether to apply filtering to the data
        char startFiltering;
        std::cout << "\nWould you like to start filtering? (y/n): ";
        std::cin >> startFiltering;

        if (startFiltering == 'y' || startFiltering == 'Y')
        {
            // Prompt the user to enter the start and end dates for
filtering
            std::string startDate = promptDate(" Enter start date
(inclusive, e.g., 2018, 1990-02, 2019-12-22): ");
            std::string endDate = promptDate("Enter end date (exclusive,
e.g., 2018, 1990-02, 2019-12-22): ");
```

```
            // Prompt the user to select the frequency for plotting
            char freq = promptFrequency();

            // Compute candlestick data based on the chosen frequency
            std::vector<Candlestick> candlesticks;
            if (freq == 'd')
            {
                candlesticks = parser.computeDailyCandlesticks();
            }
            else if (freq == 'm')
            {
                candlesticks = parser.computeMonthlyCandlesticks();
            }
            else if (freq == 'y')
            {
                candlesticks = parser.computeYearlyCandlesticks();
            }
            else
            {
                std::cerr << "Invalid frequency entered. Exiting.\n";
                return;
            }

            // Filter the candlestick data based on the specified date
range
            auto filteredCandlesticks =
Filters::filterByDateRange(candlesticks, startDate, endDate);

            // Prompt the user to specify how many entries to display
            int numEntries;
            std::cout << " How many entries do you want to display? (Enter
0 to display all): ";
            std::cin >> numEntries;
```

```
            // Limit the number of displayed entries to the specified
number

            if (numEntries == 0 || numEntries >
filteredCandlesticks.size())

            {

                numEntries = filteredCandlesticks.size();

            }


            // Resize the filtered candlestick data to the desired number
of entries

            filteredCandlesticks.resize(numEntries);


            // Display the filtered data

            std::cout << "\nOpen, High, Low, and Close values for the
selected range:\n";

            std::cout << std::setw(12) << "Date"

                      << std::setw(12) << "Open/°C"

                      << std::setw(12) << "Close/°C"

                      << std::setw(12) << "High/°C"

                      << std::setw(12) << "Low/°C" << std::endl;


            for (const auto &candle : filteredCandlesticks)

            {

                candle.displayRow();

            }


            // Plot the filtered data

            plotMultipleGraphs(filteredCandlesticks, 20, freq);


            // Display the total number of entries plotted

            std::cout << "\nTotal entries plotted: " <<
filteredCandlesticks.size() << "\n";


            // Ask the user if they want to apply additional filters after
displaying the filtered data
```

```
            char moreFilters;
            std::cout << "\nWould you like to apply more filters? (y/n): ";
            std::cin >> moreFilters;

            if (moreFilters == 'y' || moreFilters == 'Y')
            {
                // If the user wants more filters, call the
applyAdditionalFilters function
                applyAdditionalFilters(filteredCandlesticks, freq);
            }
            else
            {
                // If the user does not want more filters, display an exit
message
                std::cout << "\nExiting filtering process.\n";
            }

            // Prompt the user to decide whether they want to predict
future temperatures
            std::cout << "Would you like to predict future temperatures?
(y/n): ";
            char predictChoice;
            std::cin >> predictChoice;

            // Check the user's response to the prediction prompt
            if (predictChoice == 'y' || predictChoice == 'Y')
            {
                // If the user chooses to predict, prompt them to select
the prediction type
                std::cout << "Select prediction type: (d = days, m =
months, y = years): ";
                char periodType;
                std::cin >> periodType;

                // Variables to store the moving average window size and
the number of future periods to predict
```

```
                int windowSize, futurePeriods;
                std::cout << "Enter moving average window size: ";
                std::cin >> windowSize;
                std::cout << "Enter number of future periods to predict: ";
                std::cin >> futurePeriods;

                // Create a vector to hold the temperature data used for
predictions
                std::vector<Candlestick> temperatureData;

                // Determine which candlestick data to use based on the
selected prediction type
                if (periodType == 'd')
                {
                    // If the prediction type is daily, use the daily
candlestick data
                    temperatureData = parser.computeDailyCandlesticks();
                }
                else if (periodType == 'm')
                {
                    // If the prediction type is monthly, use the monthly
candlestick data
                    temperatureData = parser.computeMonthlyCandlesticks();
                }
                else if (periodType == 'y')
                {
                    // If the prediction type is yearly, use the yearly
candlestick data
                    temperatureData = parser.computeYearlyCandlesticks();
                }

                // Call the predictFutureTemperatures function to generate
predictions
                auto predictions =
predictFutureTemperatures(temperatureData, windowSize, futurePeriods,
periodType);
```

```
                // Display the predicted data in a table format
                std::cout << "\nPredicted Data:\n";
                std::cout << std::setw(12) << "Date"
                          << std::setw(12) << "Open/°C"
                          << std::setw(12) << "Close/°C"
                          << std::setw(12) << "High/°C"
                          << std::setw(12) << "Low/°C" << std::endl;

                // Iterate over the predictions and display each row
                for (const auto &pred : predictions)
                {
                    pred.displayRow();
                }

                // Plot the predicted data using the plotMultipleGraphs
function
                plotMultipleGraphs(predictions, 20, periodType);
            }
        }
    }
};

#endif // WEATHERANALYZER_H
/*own code end
```